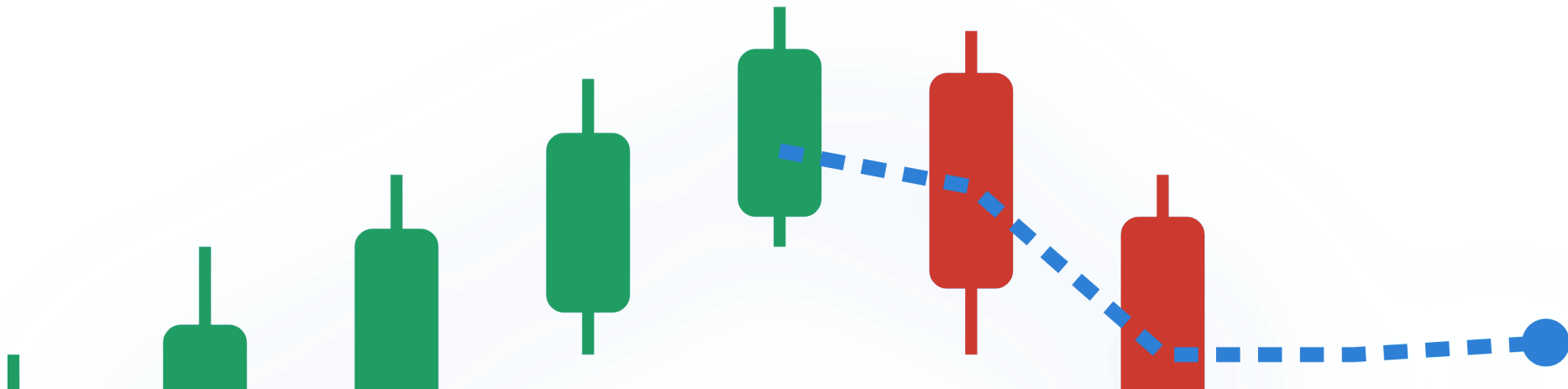


market data, GPU serving, and action — in **seconds, not batches**. This is how it runs on Azure & Microsoft Fabric.

45+ exchanges

OHLCV K-lines

real-time, not batch

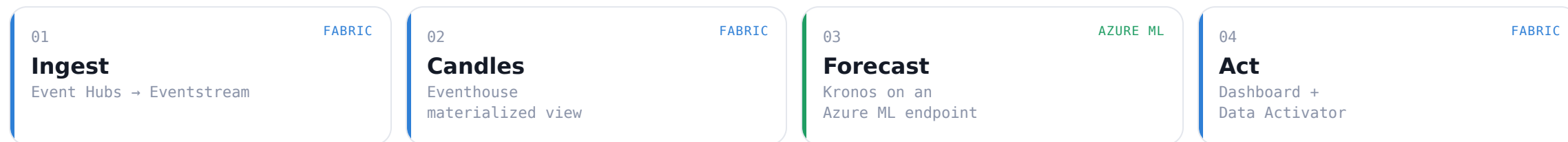


THE ARCHITECTURE

Two planes. One streaming loop.

Microsoft Fabric owns **data, time and action**; Azure ML owns **the model**.
Every candle flows through one managed pipeline.

■ MICROSOFT FABRIC – RTI · ■ AZURE ML – MODEL PLANE



Shared substrate: **OneLake** – one copy of data & models, read by every engine. Secured with Entra ID & managed identities.

Forecast in-regime — and know the instant it breaks.

SIGNALS THAT PAY

~67.6%

Directional accuracy

30-minute horizon, in-regime. Momentum calls become Data Activator signals — most of them right.

HONEST ABOUT LIMITS

0

Shocks predicted

No model forecasts an unscheduled news shock. The forecast keeps projecting the trend while price dives — as it should.

THE GUARDRAIL

~17 min

To auto-detect the break

Forecast-vs-reality error breaches its learned band and fires automatically — a regime change flagged before a human notices.

LIVE DEMO

See it run.

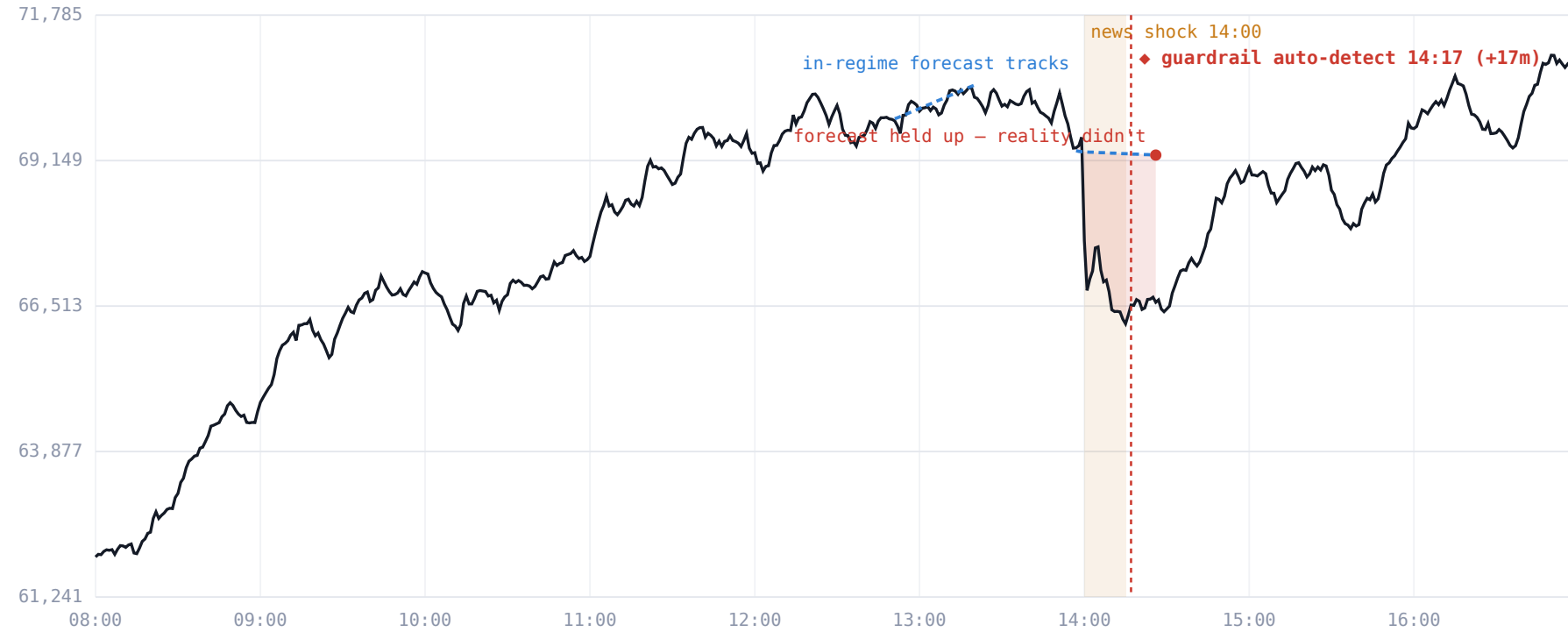
A synthetic **BTCUSDT** session runs the whole pipeline end-to-end: **6,480 ticks → 540 candles**, a Kronos forecast on every candle (140 rolling runs, 30-min horizon), and an automatic model-health guardrail. The next slides are the actual output.

Forecaster: Baseline (drift + AR(1)) — Kronos-compatible interface. Fixed-seed synthetic data — illustrative of the pipeline, not a live-market performance claim.

Price & forecast — BTCUSDT

The in-regime forecast tracks; the one issued just before the **14:00** shock (red divergence) is exactly what the guardrail is built to catch.

— actual - - - forecast ■ error



67.6%

DIRECTIONAL ACCURACY

0.54%

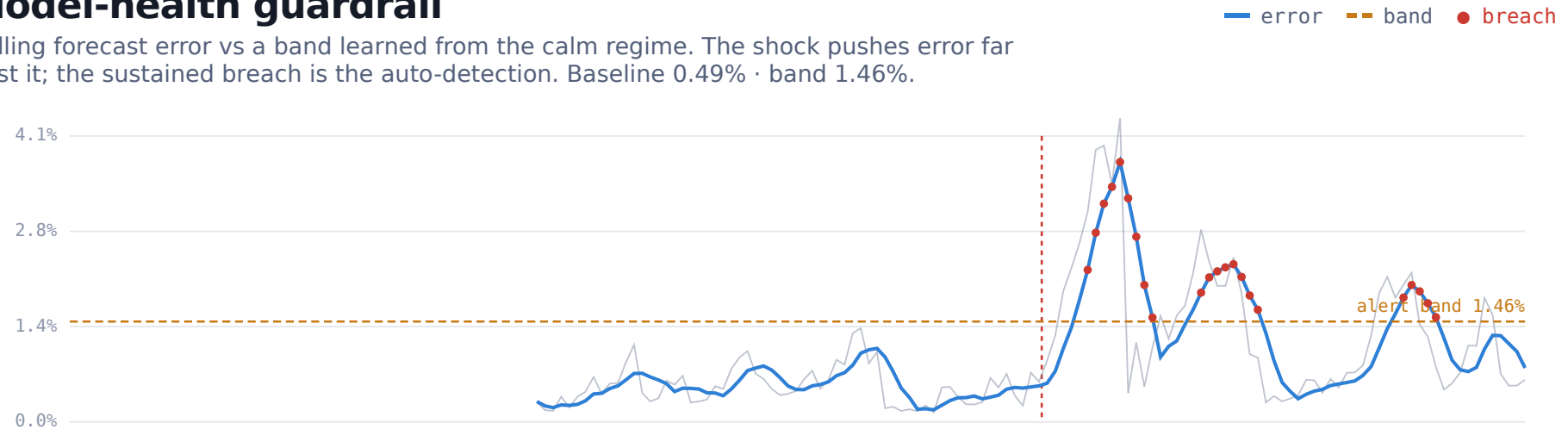
MEDIAN ERROR (MAPE)

67.9%

SIGNAL HIT-RATE · 56

Model-health guardrail

Rolling forecast error vs a band learned from the calm regime. The shock pushes error far past it; the sustained breach is the auto-detection. Baseline 0.49% · band 1.46%.



+17 min

TO AUTO-DETECT THE SHOCK

14:00

NEWS SHOCK

ACT ON IT

Momentum signals become actions.

Data Activator raises a signal when the forecast expects a move over the horizon.
Sample from the rally — overall 56 signals at 67.9% hit-rate.

TIME	CALL	EXPECTED	REALIZED	RESULT
12:20	UP	+1.31%	-0.54%	· miss
12:23	UP	+1.29%	-0.93%	· miss
12:26	UP	+1.23%	+0.28%	✓ hit
12:29	UP	+1.28%	+0.14%	✓ hit
12:38	UP	+1.10%	+0.63%	✓ hit
12:41	UP	+1.04%	+1.01%	✓ hit
12:56	UP	+1.10%	+0.31%	✓ hit

FABRIC - NATIVE

The whole run round-trips as KQL.

Every stage is emitted as `.set-or-append` against the real Eventhouse schemas. Paste one script into a Fabric queryset and the candles view, forecasts, signals and alerts all rebuild — then `deploy_to_fabric.py` does it for you.

00_setup.kql

10_replay_ticks.kql

15_candles_view.kql

20_forecasts.kql

30_signals.kql

40_model_health_alerts.kql

90_verify.kql

replay_all.kql

```
.set-or-append forecasts <|
datatable(symbol:string, run_id:string, run_time:datetime, target_time:datetime,
          open:real, high:real, low:real, close:real, ..., sampling:dynamic)
[
  "BTCUSDT", "BTCUSDT-r00120", datetime(...), 67136.3, 67232.94, 67098.44, 67195.08, 1, "baseline@demo", dynamic({"T":1.0, "top_p":0.9, "top_k":0, "sample_count":1}),
  "BTCUSDT", "BTCUSDT-r00120", datetime(...), 67195.08, 67281.76, 67157.19, 67243.87, 2, "baseline@demo", dynamic({"T":1.0, "top_p":0.9, "top_k":0, "sample_count":1}),
  ...
]
```


IN ONE LINE

Ingest → forecast → act. One managed, observable loop.

Provision & run in Fabric: `python deploy_to_fabric.py`

Fabric RTI owns data, time and action; Azure ML owns the model. Same **forecasts** schema whether the engine is the baseline or the real Kronos model.